

UNIDADE 1

**INTRODUCCIÓN Á PROGRAMACIÓN
ELEMENTOS DA LINGUAXE JAVA**

v1.2

PROGRAMACIÓN

CICLO SUPERIOR DE DAM

Manuel Pacior Pérez



Índice

1 INTRODUCCIÓN.....	5
1.1 Como crear un programa.....	5
1.2 Algoritmo e programa.....	6
1.3 Representación de algoritmos.....	8
1.3.1 Diagramas de fluxo.....	8
1.3.2 Pseudocódigo.....	8
1.3.3 Táboas de decisión.....	9
1.3.4 Exemplo: Diagrama de fluxo e pseudocódigo.....	9
1.3.4.1 Diagrama de fluxo.....	9
1.3.4.2 Pseudocódigo.....	9
2 PARADIGMAS DA PROGRAMACIÓN.....	10
2.1 Programación declarativa.....	10
2.1.1 Programación funcional.....	11
2.2 Programación imperativa.....	11
2.2.1 Programación non estruturada.....	11
2.2.2 Programación estruturada.....	11
2.2.3 Programación procedemental.....	12
2.2.4 Programación orientada a obxectos.....	12
2.2.5 Programación orientada a eventos.....	12
3 FASES DA PROGRAMACIÓN.....	12
3.1 Análise.....	13
3.2 Deseño.....	13
3.3 Implementación.....	14
3.4 Probas.....	14
3.5 Mantemento.....	15
4 LINGUAXES DE PROGRAMACIÓN.....	15
4.1 Gramática.....	16
4.2 Tipos de linguaxes.....	16
4.2.1 Linguaxe Máquina.....	16
4.2.2 Linguaxe Ensamblador.....	17



4.2.3 Linguaxes compilados.....	17
4.2.3.1 Compilador.....	18
4.2.4 Linguaxes interpretados.....	18
4.2.4.1 Intérprete.....	19
5 A LINGUAXE DE PROGRAMACIÓN JAVA.....	19
5.1 Introducción.....	19
5.2 Características.....	20
5.3 Compoñentes de Java: JDK, JRE e JVM.....	21
5.3.1 JDK.....	21
5.3.2 JRE.....	22
5.3.3 JVM.....	22
5.4 Contornos de Java.....	22
5.4.1 Java SE.....	22
5.4.1.1 Java Embedded.....	23
5.4.2 Java EE.....	23
5.4.3 Java Card.....	23
5.4.4 Java FX.....	24
5.5 Java e bytecode.....	24
5.6 Oracle JDK vs OpenJDK.....	25
5.7 Instalar o kit de desenvolvemento Java (JDK).....	25
5.8 Afinando a configuración de Java no sistema.....	26
6 ESTRUCTURA DE PROGRAMAS EN JAVA.....	26
6.1 A API de Java.....	28
6.2 Compilación e execución por terminal.....	29
7 DATOS E OPERADORES EN JAVA.....	30
7.1 Variables.....	30
7.1.1 Tipos de erro no uso de variables.....	32
7.2 Tipos de datos.....	33
7.2.1 Tipos primitivos.....	33
7.2.1.1 Números enteiros.....	34
7.2.1.2 Números en punto flotante.....	34
7.2.1.2.1 A precisión en números en punto flotante.....	34
7.2.1.3 Desbordamentos.....	35



7.2.1.3.1 Desbordamento en compilación.....	35
7.2.1.3.2 Desbordamento en execución.....	36
7.2.1.4 Caracteres.....	36
7.2.1.5 Valores lóxicos.....	37
7.2.2 Tipos por referencia.....	37
7.3 Literais.....	38
7.3.1 Literais de tipos primitivos.....	38
7.3.2 Literais de tipo String.....	39
7.4 Constantes.....	40
7.5 Operadores.....	41
7.5.1 Operadores aritméticos.....	41
7.5.2 Operadores relacionais.....	42
7.5.3 Operadores lóxicos.....	43
7.5.4 Operadores bitwise.....	44
7.5.5 Operadores de asignación compostos.....	44
7.5.6 Precedencia de operadores.....	45
7.6 Conversións de tipos.....	47
7.6.1 Conversión de ampliación (widening-conversion).....	47
7.6.2 Conversión de tipos incompatibles (casting).....	48
7.6.3 Normas de reparto e promoción de tipos.....	49
8 API DE JAVA.....	50
8.1 Saída por consola.....	51
8.2 Entrada por consola.....	55



1 INTRODUCCIÓN

Nesta primeira unidade faremos un percorrido polos conceptos fundamentais da programación de aplicacións. Comezaremos o noso camiño sabendo con que imos traballar, que técnicas podemos utilizar e que pretendemos conseguir. Continuaremos coa análise das diferentes formas de programación existentes, identificaremos que fases conforman o desenvolvemento dunha aplicación de software e avanzaremos detallando as características relevantes de cada unha das linguaxes de programación dispoñibles, para posteriormente ofrecer unha visión xeral da programación da linguaxe de programación Java. Por último, teremos a oportunidade de coñecer que ferramentas poderíamos empregar para desenvolver os nosos programas, nesa tecnoloxía.

1.1 Como crear un programa

Polo xeral, o primeiro motivo que move a unha persoa a aprender a programar é utilizar o ordenador como ferramenta para resolver problemas concretos. Como na vida real, a procura e a obtención dunha solución a un determinado problema, mediante medios informáticos, realízase seguindo uns pasos fundamentais. Na seguinte táboa podemos ver estas analoxías.

NA VIDA REAL	EN PROGRAMACIÓN
Observación da situación ou problema.	Análise do problema: é necesario que o problema estea claramente definido e comprendido para poder analízalo en detalle.
Pensamos nunha ou varias posibles solucións.	Deseño ou desenvolvemento de algoritmos: establécese unha solución ao problema sen entrar en detalles tecnolóxicos. Aplícanse diferentes técnicas e principios para establecer con detalle os pasos a seguir para resolver o problema.
Aplicamos a solución que consideremos máis axeitada.	Resolución do algoritmo elixido no ordenador: consiste en converter o algoritmo (xeito de resolver o problema) nun programa, executalo e verificar que realmente fai o que se espera.

Que características debe ter a nosa solución?

- ✓ **Corrección:** resolve o problema correctamente.
- ✓ **Eficacia:** resolve o problema nun tempo mínimo e cun uso óptimo dos recursos do sistema.



Para acadalo, cando nos enfrontemos á construción da solución, teremos que ter en conta os seguintes conceptos:

- **Abstracción:** consiste en realizar unha análise do problema para descompoñelo en problemas máis pequenos e menos complexos, describindo cada un deles con precisión. Divide e vencerás, esta adoita considerarse unha filosofía xeral para resolver problemas e, por iso, o seu nome non só forma parte do vocabulario informático, senón que tamén se utiliza en moitas outras áreas da vida. Por exemplo: nun restaurante o encargado define que debe facer cada membro do equipo, organiza como e cando debe resolver cada un as súas tarefas para que todo funcione correctamente, e a suma do traballo de todos/as constitúe a solución ao problema, neste caso dar o mellor servizo posible aos clientes.
- **Encapsulamento:** consiste en ocultar a información que manexan os distintos elementos que compoñen o sistema. A forma en que se xestiona a información non debe influír no resto dos elementos do sistema. Por exemplo: seguindo co símil do restaurante, onde cada membro do equipo ten a súa función. O camareiro non ten porque coñecer en detalle como traballa a xefa da cociña ou o encargado da limpeza para poder desempeñar correctamente a súa función.
- **Modularidade:** un proxecto de software dividirase en módulos independentes, segundo o seu tamaño, onde cada un deles terá a súa función correspondente. Os demais módulos do sistema poderán utilizar a súa funcionalidade sen necesidade de coñecer como funciona internamente.

Roger Pressman: "O principio da sabedoría para un enxeñeiro de software é recoñecer a diferenza entre facer que un programa funcione e facelo funcionar correctamente".

1.2 Algoritmo e programa

Despois de analizar en detalle o problema a resolver, debemos comezar a fase de deseño e desenvolver o algoritmo axeitado para resolvelo. Pero que é un algoritmo? Podemos definilo como unha secuencia ordenada de pasos, descritos sen ambigüidades, que conducen á solución dun problema dado.

Os algoritmos son independentes das linguaxes de programación e dos ordenadores nos que se executan. O mesmo algoritmo pódese expresar en diferentes linguaxes de programación e pode executarse en diferentes dispositivos. Pense nunha receita de cociña, pódese expresar en galego, castelán ou inglés, pode ser cociñada nun fogón ou vitrocerámica, por un ou varios cociñeiros, etc. Pero independentemente de todas estas



circunstancias, o prato elaborárase seguindo os mesmos pasos (o algoritmo).

A diferenza fundamental entre algoritmo e programa é que, no segundo, os pasos que permiten resolver o problema deben estar escritos nunha determinada linguaxe de programación para poder executalos no ordenador e obter así a solución.

As linguaxes de programación son só un medio para expresar o algoritmo, é dicir, establecen unha serie de regras sintácticas e semánticas para expresalo. O deseño dos algoritmos será unha tarefa que requirirá da creatividade do desenvolvedor e do coñecemento das técnicas de programación. Distintos estilos, de diferentes programadores á hora de obter a solución do problema, darán lugar a diferentes algoritmos igualmente válidos.

En esencia, cada problema pódese describir mediante un algoritmo e as características fundamentais que deben reunir son:

- ✓ **Preciso:** debe indicar paso a paso a orde de realización.
- ✓ **Definido:** se vai executarse dúas ou máis veces cos mesmos datos de entrada, deberá obter o mesmo resultado en cada execución. Ademais, debe proporcionar unha resposta a calquera dato introducido.
- ✓ **Finito:** Debe ser finito, é dicir, ter un número finito de pasos.

A continuación imos amosar como se podería expresar un algoritmo para preparar unha infusión.

Entrada: cazo, cunca, bolsa de infusión
Saída: cunca coa infusión

Inicio

Coller o cazo
Enchelo de auga
Acender o lume
Colocar o cazo no lume
Agardar a que ferva a auga
Coller a bolsa de infusión
Poñer a bolsa na cunca
Agardar os minutos recomendados
Retirar a bolsa da cunca

Fin

O exemplo anterior é moi simple, ademais trátase dun algoritmo de alto nivel, xa que non entra en detalle de como se fai cada unha das accións, nin que faríamos ante



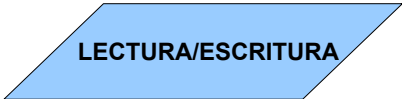
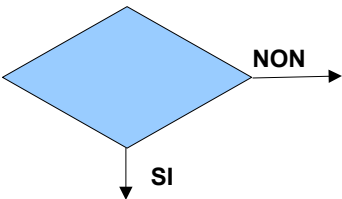
determinadas situacións (condicións).

1.3 Representación de algoritmos

Para representar graficamente os algoritmos que imos deseñar, temos á nosa disposición diferentes ferramentas que nos axudarán a describir o seu comportamento de forma precisa e xenérica, para logo codificalos coa linguaxe que nos interese. As máis estendidas son as seguintes.

1.3.1 Diagramas de fluxo

Esta técnica utiliza símbolos gráficos para representar o algoritmo e adoita empregarse nas fases de análise. Cada operación está representada por un símbolo estandarizado polo ANSI (American National Standards Institute). Os símbolos máis comúns son:

SÍMBOLO		DESCRICIÓN
	Inicio/Fin	Úsase para indicar o inicio/fin do algoritmo
	Liña de fluxo	Indica a orde na que se executan as operacións. A frecha indica a seguinte instrución.
	Proceso	Representa calquera tipo de operación a executar.
	Entrada/saída	Representa a lectura de datos na entrada e a impresión de datos na saída.
	Decisión	Permite analizar unha situación, e en base a iso tomar unha decisión.

1.3.2 Pseudocódigo

Esta técnica baséase no uso de palabras clave en linguaxe natural, constantes, variables, outros obxectos, instrucións e estruturas de programación que expresan a solución do problema en forma escrita, pero dun xeito informal, sen seguir as restricións semánticas de ningunha linguaxe.



1.3.3 Táboas de decisión

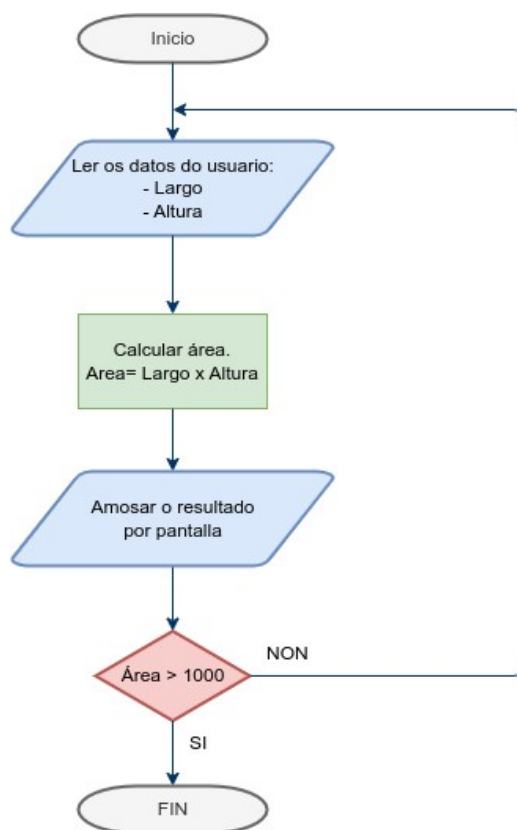
Unha táboa de decisión representa as posibles condicións do problema coas súas respectivas accións. Comentamos a súa existencia, máis non imos usalas neste módulo.

1.3.4 Exemplo: Diagrama de fluxo e pseudocódigo

Imaxinemos que queremos realizar unha pequena aplicación que permita ao usuario inserir o tamaño dos lados dun rectángulo (largo ou ancho e altura) para calcular a súa área. Ademais debemos ter en conta que o programa so finalizará cando a area do rectángulo calculado sexa maior de 1000.

1.3.4.1 Diagrama de fluxo

A continuación indicamos unha posible solución, representando o algoritmo a través dun diagrama de fluxo.



1.3.4.2 Pseudocódigo

Inicio

Repetir

Escribir “Insire a altura:”

Ler altura



```
    Escribir "Insire o ancho:"  
    Ler ancho  
    area = altura x ancho  
    Mostrar area por pantalla  
    Até que area > 1000  
Fin
```

2 PARADIGMAS DA PROGRAMACIÓN

Supoño que estás pensando cantas formas hai de facer as cousas: varias (ou incluso moitas). Pero cando se establece un patrón para a creación de aplicacións estamos cada vez máis preto do significado da palabra paradigma.

Se establecemos unha serie de regras e principios que recollen experiencias e boas prácticas doutros desenvolvedores para o seu uso na resolución de problemas, estaremos creando un paradigma de programación.

Podemos polo tanto definir **paradigma de programación** como o enfoque ou filosofía particular para a construción de software. Cada un terá as súas vantaxes e inconvenientes, será máis ou menos axeitado, pero non é correcto dicir que hai un mellor que os outros.

Existen múltiples paradigmas, incluso pode haber linguaxes de programación que non se clasifiquen só nun deles. Por exemplo:

- **Java**: é unha linguaxe baseada no paradigma orientado a obxectos, que tamén admite outros, como a orientada a eventos.
- **Scheme**: só admite programación funcional
- **Python** admite múltiples paradigmas.

2.1 Programación declarativa

Baséase no desenvolvemento de algoritmos aplicando unha especificación dun conxunto de condicións, proposicións, enunciados e restricións que describen o problema. As frases empregadas describen o problema que se quere resolver, pero non as instrucións necesarias para chegar á solución.

A linguaxe SQL baséase neste paradigma, do que tamén forman parte a programación funcional e a programación lóxica.



2.1.1 Programación funcional

Esta baseada na definición dos predicados, de corte matemático. Este paradigma, por non conter datos mutables, caracterízase por estar indicado para a xestión da información, máis non para a creación ou modificación da mesma.

Por exemplo:

```
Long result = numeros.stream().filter(num -> num > 10).count();  
System.out.println(result);
```

2.2 Programación imperativa

Ten como obxectivo o desenvolvemento de algoritmos que detallan de forma clara e específica os comandos a executar para, a través do paso por diferentes estados, chegar á solución. Baséase no uso de variables, tipos de datos, expresións e estruturas de control de fluxo de execución.

A programación imperativa baséase na máquina de Turing mentres que a programación declarativa baséase no cálculo lambda. Linguaxes como Python, Java, C++, C#... fan uso deste paradigma, aínda que tamén soportan a programación funcional. Dentro deste paradigma podemos incluír os seguintes.

2.2.1 Programación non estruturada

A programación non estruturada foi a utilizada nos albores da programación, carecía de estruturas de fluxo de control, e os saltos entre as partes do código eran realizada facendo a popular instrución "GoTo", que saltaba á liña de código do programa indicada. Este paradigma producía o que se coñecía como código spaghetti, e dificultaba a creación de grandes aplicacións, así como do seu mantemento.

2.2.2 Programación estruturada

A programación estruturada é un paradigma de programación destinado a mellorar a claridade, calidade e o tempo de desenvolvemento dun programa informático. Para iso, bota man de:

- Funcións ou subrutinas.
- Estruturas básicas de control:
 - Secuencia.
 - Selección (if e switch).



- Iteración (bucles for e while).

Trátase dunha evolución da primitiva programación non estruturada, porén o uso da rutina GOTO considérase innecesaria e contraproducente.

2.2.3 Programación procedemental

O enfoque procedemental divide as tarefas que debe xestionar un programa en tarefas parciais máis pequenas. Deste xeito, créanse principios básicos de programación que tamén poden ser reutilizados noutros programas.

Por exemplo:

```
function float calcArea( float base, float altura ) {  
    return (base*altura)/2  
}  
.  
.  
.  
float area1 = calcArea(23f, 12f);  
float area2 = calcArea(3f, 27f);
```

2.2.4 Programación orientada a obxectos

A Programación Orientada a Obxectos (POO) é un paradigma de programación que se basea no concepto de clases e obxectos. Este tipo de programación utilízase para estruturar un programa de software en pezas simples e reutilizables.

Trátase dunha evolución sobre a programación estruturada, que optimiza e simplifica a reutilización do código, e facilita o seu mantemento.

2.2.5 Programación orientada a eventos

Trátase dun paradigma de programación no que tanto a estrutura como a execución de dous programas están determinadas por eventos que ocorren no sistema, definidos polo usuario ou que eles mesmos provocan.

Hai linguaxes como Java que aínda estando orientado a obxectos, soportan que a nosa aplicación defina tamén ese comportamento. Por exemplo, Swing e **JavaFX**, as librerías para traballar con interfaces gráficas de usuario, traballan usando este paradigma.

3 FASES DA PROGRAMACIÓN

Sexa cal sexa o estilo que escollemos á hora de automatizar unha determinada tarefa, debemos levar a cabo o proceso aplicando unha metodoloxía ao noso traballo. Coñecemos



que imos resolver un problema aplicando unha filosofía de desenvolvemento, e farémolo dando unha serie de pasos que deben estar ben definidos e organizados.

O proceso de creación de software pódese dividir en diferentes fases:

- ✓ Análise.
- ✓ Deseño.
- ✓ Implementación (codificación).
- ✓ Probas.
- ✓ Instalación e paso a produción.
- ✓ Mantemento.

A continuación, analizaremos cada un delas.

3.1 Análise

Normalmente, a análise indicará a especificación dos requisitos que se deben cubrir. Os contactos entre o analista/programador e o cliente/usuario serán numerosos, deste xeito poderán coñecerse todas as necesidades que require a aplicación. Especificaranse os procesos e estruturas de datos a utilizar. A creación de prototipos será moi útil para coñecer con máis exactitude os puntos a tratar.

A análise inicial ofrecerá unha idea xeral do que se solicita, realizando posteriormente perfeccionamentos sucesivos que servirán para dar resposta ás seguintes cuestións:

- ✓ Cal é a información que ofrecerá a resolución do problema?
- ✓ Que datos son necesarios para resolver o problema?

A resposta á primeira pregunta identifícase cos resultados desexados ou solucións ao problema. A resposta da segunda pregunta indicará que datos se proporcionan, ou cales son as entradas para o problema.

Nesta fase debemos aprender a analizar a documentación da empresa, investigar, observar todo o que rodea o problema e recoller calquera información útil.

3.2 Deseño

Nesta etapa, a especificación definida na fase de análise convértese nun deseño máis detallado, indicando o comportamento ou a secuencia lóxica de instrucións capaces de resolver o problema proposto. Estes pasos sucesivos, que indican as instrucións que debe executar a máquina, constitúen o que coñecemos como algoritmo.



Durante a fase de deseño, a aplicación que se vai desenvolver considérase como unha única operación global, e descomponse en operacións máis sinxelas, detalladas e específicas. O nivel de descomposición dependerá do tamaño do problema. En cada nivel de refinamento, as operacións identificadas son asignadas a módulos separados.

Hai que ter en conta que antes de pasar á implementación do algoritmo, debemos asegurarnos de ter unha solución adecuada. Se a solución aportada presenta erros, teremos que volver á fase de análise para realizar as modificacións necesarias ou tomar un novo camiño para a solución. Só cando o algoritmo deseñado cumpra os requisitos e obxectivos especificados na fase de análise, pasará á fase de implementación.

3.3 Implementación

Se as anteriores fases realización da análise e posterior deseño da solución son moi importantes para obter un produto que cumpra ás necesidades definidas polo cliente dun xeito eficiente, a implementación tórnase nunha fase crítica e fundamental.

Esta etapa consiste en transformar ou traducir os resultados obtidos a unha linguaxe de programación específica. Cando traducimos o algoritmo á linguaxe de programación, debemos ter en conta as regras gramaticais e a sintaxe da que soporta, usando a arquitectura de software definida. Iso daranos o código fonte da aplicación, que normalmente coñecemos como programa.

3.4 Probas

Nesta fase debemos comprobar a calidade e estabilidade da aplicación, e para iso débense realizar unha serie de probas, entre as que destacamos:

- ➔ **Probas unitarias:** Verificar o correcto funcionamento das funcións de cada módulo.
- ➔ **Probas de integración:** Validar que os módulos funcionan correctamente en conxunto, é dicir, que se integran ben entre eles.
- ➔ **Probas funcionais:** Céntranse en validar que a aplicación cumpre os requirimentos empresarias definidos.
- ➔ **Probas de rendemento:** As probas de rendemento avalían o rendemento dun sistema cunha carga de traballo determinada

Para esta etapa é necesario implantar a aplicación nun sistema similar onde vai funcionar, para iso adoitan crearse entorno de proba, que son un copia do entorno de produción (que é onde se vai executar realmente). Debe iniciarse e comprobar se o seu



funcionamento é correcto, usando para elo diferentes datos de proba, cos que se comprobarán os seguintes aspectos:

- Se o programa responde aos requisitos especificados.
- Se se detectan novos erros e están ben xestionados.
- Se a interface é amigable.

En definitiva, trátase de probar o noso programa para ver a súa resposta en situacións difíciles.

Unha vez corrixido e probado o programa, actualizarase a documentación, que pode ser:

- ➔ **Documentación interna:** cabeceiras, descrições, declaracións de problemas e comentarios incluídos no propio código fonte (código da aplicación na linguaxe de programación codificada).
- ➔ **Documentación externa:** Son os manuais que se crean para unha mellor execución e uso do programa.

3.5 Mantemento

Cando o programa xa está instalado no sistema e está sendo útil aos usuarios, dicimos que está en fase de explotación.

Periodicamente será necesario realizar avaliacións e, de ser o caso, realizar modificacións para que o programa se adapte ou se actualice ás novas necesidades, e tamén se poidan corrixir erros non detectados previamente. Este proceso chámase mantemento de software.

Para realizar correctamente esta fase, será fundamental xerar unha documentación adecuada, que facilite ao programador a comprensión, uso e modificación dese programa.

4 LINGUAXES DE PROGRAMACIÓN

Podemos definir unha linguaxe de programación con regras sintácticas e semánticas, símbolos e palabras especiais establecidos para a construción de programas. Trátase dunha linguaxe artificial, unha construción mental do ser humano para crear programas (conxunto de instrucións).

As linguaxes de programación, tal como acontece con outro tipo de linguaxes, basea o seu funcionamento nunha gramática.



Hai que ter en conta que pode haber frases sintacticamente correctas, pero semanticamente incorrectas. Por exemplo, "Un porco ladrou durante toda a noite" está ben construído a nivel sintáctico, malia que non está ben construído semanticamente (significado).

Unha característica relevante das linguaxes de programación é precisamente que máis dun programador pode usar un conxunto común de instrucións que se entenden entre si. A través deste conxunto podemos construír aplicacións de xeito colaborativo.

As linguaxes de programación pódense clasificar en función da proximidade que estean da linguaxe humana ou da linguaxe das computadoras. Trátase de códigos binarios, é dicir, secuencias de uns e ceros que representan instrucións para que se executan no sistema operativo da computadora.

4.1 Gramática

Son todas aquelas normas aplicables ao conxunto de símbolos e palabras da linguaxe de programación, utilizadas para a construción de oracións correctas.

Cada gramática ten:

- **Léxico:** é o conxunto finito de símbolos e palabras especiais, trátase do vocabulario da lingua.
- **Sintaxe:** Son as posibles combinacións de símbolos e palabras especiais. Está relacionado coa forma dos programas.
- **Semántica:** É o significado de cada construción lingüística, a acción que se vai levar a cabo con ela.

4.2 Tipos de linguaxes

Agora detallaremos os principais tipos de linguaxes de programación.

4.2.1 Linguaxe Máquina

Esta é a linguaxe utilizada directamente polo procesador, consta dun conxunto de instrucións codificadas en binario. É o sistema de código directamente interpretable por un circuío microprogramable.

Esta foi a primeira linguaxe utilizada para a programación de ordenadores. De feito, cada máquina tiña o seu propio conxunto de instrucións codificadas en ceros e uns. Cando se escribe un algoritmo neste tipo de linguaxe, dicimos que está en código máquina.

A programación neste tipo de linguaxe presentaba os seguintes inconvenientes:



- x Cada programa era válido só para un tipo de procesador ou ordenador.
- x Ler ou interpretar os programas era moi difícil e, polo tanto, inserir modificacións era moi custoso.
- x Os programadores da época tiñan que memorizar longas combinacións de ceros e uns, que equivalían ás instrucións dispoñibles para os distintos tipos de procesadores.
- x Os programadores estaban encargados de introducir os códigos binarios no ordenador, o que provocou longos tempos de preparación e posibles erros.

4.2.2 Linguaxe Ensamblador

A evolución da linguaxe máquina foi a linguaxe ensamblador. As instrucións xa non son secuencias binarias, son substituídas por códigos de operación que describen un funcionamento elemental do procesador.

Trátase aínda dunha linguaxe de baixo nivel, como a linguaxe máquina, xa que depende directamente do hardware no que se execute. Normalmente, unha instrución en ensamblador correspóndese a unha instrución de linguaxe máquina.

Aínda que a linguaxe ensamblador foi un intento de achegar a linguaxe dos procesadores á linguaxe humana, presentaba aínda múltiples dificultades:

- x Os programas seguiron dependendo directamente do hardware que os admitía.
- x Os programadores debían coñecer polo miúdo a máquina na que estaban programando, xa que debían facer un uso axeitado dos recursos do equipo onde se executaba.
- x A lectura, interpretación ou modificación dos programas seguiu presentando dificultades.

Todo programa escrito en linguaxe ensamblador precisa dun intermediario para traducir cada unha das instrucións que compoñen o seu código á linguaxe máquina correspondente. Este intermediario é o que se coñeza como programa ensamblador. O programa orixinal escrito en linguaxe ensamblador constitúe o código fonte e o programa traducido á linguaxe máquina coñécese como programa obxecto que será executado directamente polo ordenador.

4.2.3 Linguaxes compilados

Para paliar os problemas derivados do uso da linguaxe ensamblador, e co obxectivo de



achegar a programación ao uso dunha linguaxe máis próxima ao humano que ao ordenador, naceron linguaxes compiladas. Algúns exemplos deste tipo de linguaxes son: Pascal, Fortran, Algol, C, C++, etc.

Ao ser linguas máis próximas aos humanos, tamén se chaman linguas de alto nivel. Son máis fáciles de usar e comprender, as instrucións que forman parte destas linguaxes utilizan palabras e signos recoñecibles polo programador.

Entre as súas principais vantaxes destacamos:

- ✓ Son moito máis fáciles de aprender e usar que os seus predecesores.
- ✓ Redúcese o tempo para desenvolver programas, así como os custos.
- ✓ Son independentes do hardware, os programas poden executarse en diferentes tipos de máquinas.
- ✓ Ler, interpretar e modificar os programas é moito máis sinxelo.

Como principal desvantaxe podemos destacar que polo xeral dicir o código obxecto xerado é menos eficiente que o código xerado en linguaxe ensamblador, a pesar de que os compiladores realizan procesos de optimización de código.

Aquí vemos un novo concepto, pois un programa que está escrito nunha linguaxe de alto nivel ten que ser traducido a código que a máquina poida usar, e para iso temos programas tradutores, que se denominan compiladores.

4.2.3.1 Compilador

Trátase dun programa cuxa función é traducir o código fonte dun programa escrito nunha linguaxe de alto nivel á linguaxe máquina. O proceso de tradución coñécese como compilación.

O compilador realizará a tradución e tamén informará dos posibles erros. Unha vez verificado o programa, e se non hai erros, xerarase o programa traducido a código máquina, coñecido como código obxecto. Este programa aínda non se pode executar ata que se engadan os módulos de ligazón ou as bibliotecas durante o proceso de vinculación. Unha vez rematada a ligazón, obtense o código executable.

4.2.4 Linguaxes interpretados

Caracterízanse por estar deseñados para que a súa execución se realice a través dun intérprete. Cada instrución escrita nunha lingua interpretada é analizada, traducida e executada despois de ser verificada. Unha vez que o intérprete realiza o proceso, a



instrución execútase, pero non se garda na memoria.

4.2.4.1 Intérprete

Estamos diante dun programa tradutor de idiomas de alto nivel no que o proceso de tradución e execución se realizan simultaneamente, é dicir, a instrución pásase a linguaxe máquina e execútase directamente. Non se xera ningún programa obxecto nin programa executable.

As linguaxes interpretadas xeran programas máis pequenos que os obtidos por un compilador, xa que non gardan o programa traducido en código máquina. Porén teñen a desvantaxe de ser algo máis lentos, xa que teñen que ser traducidos durante a súa execución. Por outra banda, necesitan ter o programa intérprete en execución na máquina, algo que non é necesario no caso dun programa compilado, para o que só é necesario ter o programa executable para poder utilizalo. Algúns exemplos de linguaxes interpretadas máis destacados son: Python, JavaScript, Perl ou PHP .

A medio camiño entre as linguaxes compiladas e as interpretadas, hai linguaxes que podemos chamar pseudo-compiladas ou pseudo-interpretadas, este é o caso da Linguaxe Java. Pódese ver como compilado e interpretado ao mesmo tempo, xa que o seu código fonte está compilado para obter o código binario en forma de bytecodes, que é un código intermedio con instrucións similares á do código máquina, mas coa importante vantaxe de non depender de ningún tipo de máquina.

5 A LINGUAXE DE PROGRAMACIÓN JAVA

5.1 Introducción

Java xurdiu en 1991 cando un grupo de enxeñeiros de Sun Microsystems intentou deseñar unha nova linguaxe de programación destinada a programar pequenos dispositivos electrónicos. A dificultade deses dispositivos é que mudaban continuamente e para que un programa funcionara no seguinte dispositivo que aparecía, había que reescribir o código. Por este motivo a compañía Sun decidiu deseñar unha linguaxe que fora independente do dispositivo ou plataforma onde se executara.

Non foi ata 1995 cando pasou a chamarse Java, pasando a ser coñecida polo público como unha linguaxe de programación para ordenadores. Converteuse nunha linguaxe completamente independente da plataforma, cunha grande potencia e orientada a obxectos. Esta filosofía e a súa facilidade para crear aplicacións para redes TCP/IP,



convertérona nunha das linguaxes máis utilizados na actualidade.

5.2 Características

Java é unha linguaxe fácil de aprender, cunha sintaxe similar á de C++, pero na que se eliminaron elementos complicados que poden provocar erros. Estamos diante dunha linguaxe de programación creada para satisfacer novas necesidades que as linguaxes existentes ata agora non eran capaces de resolver. Entre as súas principais características destacamos:

- ➔ Está orientada a obxectos, o que elimina moitas preocupacións para o programador e permite o uso dunha gran cantidade de bibliotecas xa definidas, evitando reescribir código que xa existe.
- ➔ Unha das súas principais vantaxes é a independencia hardware, xa que o código xerado é válido para calquera plataforma (Windows, Linux ou macOS).
- ➔ O código xerado en JAVA executarase a través dunha máquina virtual denominada Java Virtual Machine (JVM – Java Virtual Machine), que interpretará o código, converténdoo en código específico da plataforma que o soporta.
- ➔ A súa lóxica de funcionamento segue o lema: "Write once, run everywhere" (Escribe unha vez, corre por todas partes).

Antes da aparición de Java, a linguaxe C era unha das máis estendidas pola súa versatilidade. Pero cando os programas escritos en C aumentaron de volume, o seu manexo comezou a complicarse. Usando programación estruturada e técnicas de programación modular, estas complicacións reducíronse, pero non foi suficiente.

Nese momento entra en escena o paradigma da Programación Orientada a Obxectos (OOP), achegando e simplificando o proceso de construción de programas ao pensamento humano. Os problemas divídense en obxectos que teñen propiedades e interactúan con outros obxectos, deste xeito, o programador pode centrarse en cada obxecto para programar internamente os elementos e funcións que o compoñen.

As principais características da linguaxe Java resúmense a continuación:

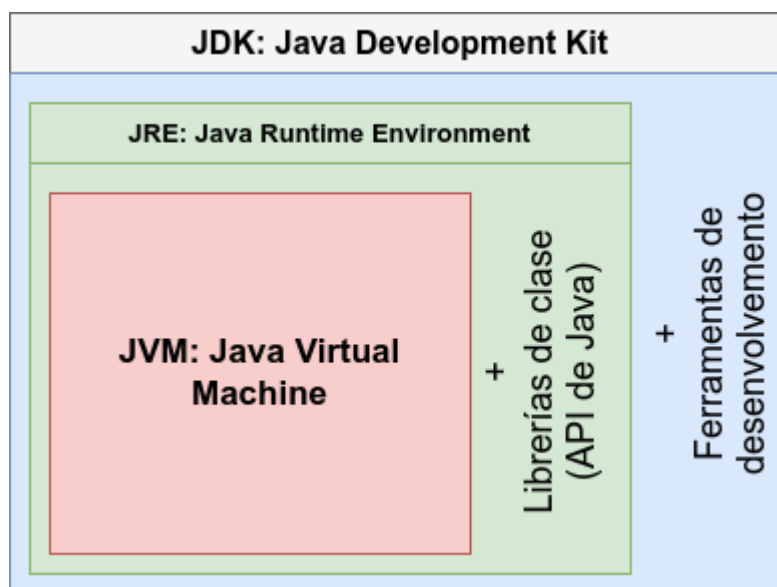
- ✓ O código xerado polo compilador Java é independente da arquitectura.
- ✓ Está completamente orientado a obxectos.
- ✓ A súa sintaxe é semellante a C e C++.
- ✓ É distribuído, isto quere dicir que está preparado para desenvolver aplicacións

TCP/IP (que se comuniquen a través de redes de ordenadores).

- ✓ Ten un amplo conxunto de bibliotecas.
- ✓ É robusto, realiza comprobacións de código en tempo de compilación e execución.
- ✓ A seguridade está garantida, xa que as aplicacións Java non acceden a áreas sensíbles da memoria ou do sistema.

5.3 Compoñentes de Java: JDK, JRE e JVM

Para traballar en Java, existen unha serie de ferramentas que imos describir en detalle neste apartado. A continuación amosamos un esquema dos compoñentes JDK, JRE e JVM, onde podemos ver como se relacionan entre si.



Imaxe. Arquitectura de JVM, JRE e JDK

5.3.1 JDK

O Kit de Desenvolvemento de Java (JDK) é un contorno de desenvolvemento de software usado para desenvolver aplicacións Java. Inclúe:

- O Java Runtime Environment (JRE).
- Un intérprete/cargador (Java).
- Un compilador (javac).
- Un arquivador (jar).
- Un xerador de documentación (Javadoc).
- Outras ferramentas necesarias para o desenvolvemento de Java.



5.3.2 JRE

O seu nome significa "Java Runtime Environment", e proporciona os requisitos mínimos para executar unha aplicación Java:

- A Java Virtual Machine (JVM).
- As bibliotecas principais.
- Outros compoñentes adicionais para executar aplicacións escritas en Java.

5.3.3 JVM

Unha máquina virtual Java (JVM) é un programa executable, para unha plataforma específica (Windows, Linux, ...), capaz de interpretar e executar instrucións expresadas nun código binario especial (o bytecode Java), que é xerado polo compilador da linguaxe Java.

Mentres que a versión de JVM executada por cada plataforma é nativa para cada unha delas, o bytecode Java xerado e interpretado por esta é común a todas elas. Por este motivo se considera aos executables xerados con Java (Bytecode) como multiplataforma.

A continuación describimos as características máis destacables da JVM:

- O código binario Java non é unha linguaxe de alto nivel, senón un verdadeiro código de máquina de baixo nivel, viable mesmo como linguaxe de entrada para un microprocesador físico.
- Como todas as pezas do crebacabezas Java, foi desenvolvido orixinalmente por Sun.
- Trátase dunha das pezas fundamentais da plataforma Java, pois sitúase a un nivel superior ao hardware do sistema no que se pretende executar a aplicación, e actúa como ponte que comprende tanto o bytecode como o sistema no que se pretende executar.

5.4 Contornos de Java

Java non para de medrar, pois é amplamente utilizada, engadindo continuamente tecnoloxías que achegan novas funcionalidades. Entre as diferentes plataformas que fornece destacamos as seguintes.

5.4.1 Java SE

Contorno Sun indicado para a implementación de aplicacións en linguaxe Java para a súa execución nos PC e servidores. Trátase da plataforma estándar, que integra todas as



funcionalidades básicas da linguaxe. Se comezas con Java, o máis normal é descargar as bibliotecas Java SE. Dentro destas funcionalidades básicas de Java destacamos:

- Uso de coleccións.
- Acceso a ficheiros con Java IO e NIO.
- Acceso a bases de datos con JDBC.
- Bibliotecas para data e hora.
- Mecanismos para a creación de aplicacións multiproceso.
- Capacidades para realizar conexións de rede.
- Xestión de contido XML.

5.4.1.1 Java Embedded

Unha plataforma aberta e intersectorial para a Internet das cousas (IoT) para a execución de aplicacións Java en sistemas embebidos.

Ofrece novas capacidades para dispositivos integrados, estando moi relacionado co popular IoT (Internet das cousas).

5.4.2 Java EE

Java Platform Enterprise Edition (anteriormente coñecida como Java 2 Platform, Enterprise Edition ou J2EE ata a versión 1.4) é unha plataforma de programación de aplicacións empresariais con Java. Permite o uso de arquitecturas distribuídas de capas N e é amplamente compatible con compoñentes de software modulares que se executan nun servidor de aplicacións. Java EE permítenos entre outras:

- Desenvolver servizos web; xa sexan WSDL (con JAX-WS) ou REST (con JAX-RS)
- Creación de websockets.
- Ofrece unha API de persistencia de obxectos con JPA.
- Permite capacidades de mensaxería con Java Message.
- Dota de capacidades de correo electrónico con Java Mail.

5.4.3 Java Card

Tecnoloxía de software que permite que as aplicacións baseadas en Java se executen de forma segura en tarxetas intelixentes e, de xeito máis xeral, en dispositivos de pequena memoria.



5.4.4 Java FX

Aínda que non é unha plataforma como tal, trátase da librería máis destacada que ten Java para o desenvolvemento de interfaces gráficas de usuario. Esta é unha librería de [código aberto](#) moi útil para implementar GUI (interfaces gráficas de usuario) en Java. Conta cun deseñador chamado [Java Scene Builder](#), que facilita a creación e edición de interfaces de usuario.

5.5 Java e bytecode

Un programa escrito en Java non é executable directamente, é necesario que o código fonte sexa interpretado pola Máquina Virtual Java. A continuación detallamos cales son os pasos seguidos desde a xeración do código fonte, ata que se executa:

1. Unha vez escrito o código fonte (arquivos con extensión .Java), pre-compílese, xerando os equivalentes en bytecode, que son ficheiros con extensión .class.
2. Os ficheiros de bytecode serán interpretados directamente pola JVM (Máquina Virtual Java) e traducidos ao código nativo da plataforma na que se está a executar o programa.

Polo tanto, podemos definir bytecode como o conxunto de instrucións en linguaxe máquina, que non son específicas de ningún procesador ou sistema informático. Un intérprete de bytecode, para unha plataforma específica, será a encargada de executalos.

No proceso de pre-compilación, hai un validador de bytecode que asegurará que se cumpran as seguintes condicións:

- ✓ O código cumpre as especificacións da máquina virtual Java.
- ✓ Non hai ningunha ameaza para a integridade do sistema.
- ✓ Non se producen desbordamentos de memoria.
- ✓ Os parámetros e os seus tipos son axeitados.
- ✓ Non hai conversións de datos que non estean permitidas.

Para que un bytecode se execute en calquera plataforma é fundamental que dita plataforma conte co intérprete axeitado, é dicir, a JVM específica para esa plataforma (GNU/Linux, Windows, macOS, ...). En xeral, a máquina virtual Java é un programa dun tamaño moi pequeno, que se pode descargar gratis para os sistemas operativos máis utilizados.



5.6 Oracle JDK vs OpenJDK

Para traballar coa tecnoloxías Java existen actualmente dúas solucións que destacan, son [Oracle JDK](#) e [OpenJDK](#). Podemos traballar con calquera das dúas.

Tanto [Oracle JDK](#) como [OpenJDK](#) son conxuntos de software e especificacións coñecidos como Java Development Kits. De feito, a partires de Java 11, ambos son compilados do mesmo xeito e inclúen funcións como [Flight Recorder](#) e [Mission Control](#). Ambos teñen basicamente o mesmo código, polo que funcionan de xeito moi semellante.

A principal diferenza entre eles é que [OpenJDK](#) é un proxecto de código aberto, mantido por varias entidades: Red Hat, Oracle e a comunidade, mentres que [Oracle JDK](#) é mantido unicamente pola empresa Oracle e ademais é de código pechado. Algunhas funcións non están dispoñibles con [OpenJDK](#) porque son de código pechado, ou requiren dunha licenza de pago.

5.7 Instalar o kit de desenvolvemento Java (JDK)

Xa coñecemos como é a estrutura básica dun programa Java, agora precisamos instalar o JDK (Java Development Kit), a ferramenta básica para comezar a desenvolver aplicacións Java, que inclúe un compilador e un intérprete de liña de comandos. Estes dous programas utilízanse na pre-compilación e interpretación do código.

Como veremos, existen diferentes contornos para a creación de programas Java que inclúen multitude de ferramentas, pero polo momento centrarémonos no entorno máis básico, estendido e libre, o Java Development Kit (JDK).

Tal como vimos, o JDK, inclúe unha implementación do contorno de execución Java, o JRE (Java Runtime Environment), e este á súa vez integra a máquina virtual Java (JVM), bibliotecas de clases e outros ficheiros que admiten a execución de programas escritos na linguaxe de programación Java.

A continuación deixo varios enlaces onde indica como instalar o JDK para calquera Windows e GNU/Linux:

➔ **Ubuntu Linux:** Instrucións para [instalar Oracle JDK e OpenJDK en Ubuntu](#)

➔ **Windows:**

- Instrucións para [instalar Oracle JDK en Windows](#)
- Instrucións para [instalar OpenJDK en Windows](#)



5.8 Afinando a configuración de Java no sistema

Para que poidamos compilar e executar ficheiros Java, necesitamos facer algúns pequenos axustes na configuración do sistema. Imos indicarche onde atopar os ficheiros necesarios para realizar as tarefas de compilación e execución, neste caso Javac.exe e Java.exe, así como as bibliotecas contidas na API de Java e as clases de usuario.

A variable PATH: Como aínda non temos un IDE (Integrated Development Environment), a única forma de executar programas é a través da liña de comandos. Pero só podemos executar programas directamente se o camiño a eles está indicado na variable PATH do ordenador. Necesitamos incluír o camiño a estes programas na nosa variable PATH. Este camiño será o lugar onde se instalou o JDK ata o seu directorio bin.

Podemos seguir as instrucións que se indican nos seguintes links:

- **Windows 10/11:** [Configurar a variable PATH en Windows 10/11](#)
- **Ubuntu:** Se realizamos unha instalación desde o xestor de paquetes, xa debería quedar configurado. Podemos probalo abrindo unha consola e executando o comando "java -version". Se está ben devolverá a versión do JDK instalado por defecto. En caso contrario, será preciso configurar a variable PATH para que conteña a ruta aos binarios (executables do directorio bin) que contén o directorio onde instalamos o JDK. No [seguinte link](#) indica como se realiza isto de xeito manual.

6 ESTRUCTURA DE PROGRAMAS EN JAVA

Ata agora, describimos a linguaxe de programación Java, fixemos un percorrido pola súa historia e coñecemos a súa filosofía de traballo.

Estamos cada vez máis preto de comezar a experiencia coa linguaxe de programación Java. Antes diso imos comezar o noso camiño descubrindo cales son os elementos básicos dun programa Java, a forma na que debemos escribir o código, así como os tipos de aplicacións que se poden crear nesta linguaxe.

No código que se mostra a continuación podemos ver a estrutura xeral dun programa nunha linguaxe orientada a obxectos como Java.

```
package prog.daw.fp;  
  
/**
```



```
* Estructura xeral dunha aplicación simple en Java
*/
public class ClasePrincipal {
    //Aquí iría a definición de atributos da clase
    //Aquí iría a definición de métodos da clase
    //Aquí iría a definición doutras (sub)clases

    /*
     * Declaración do método main, que é o punto de entrada
     * dun programa executable.
     */
    public static void main(String[] args) {
        //Declaración de variables de método
        //Instrucións que se desexen inserir aquí..
        System.out.println("Ola mundo!");
    }
}
```

No anterior código podemos ver que no exemplo anterior, aparece definida “public class ClasePrincipal”. trátase dunha clase xeral na que se inclúen todos os demais elementos necesarios para iniciar o programa. Podemos observar o método **main(String[] args)**, que representa o punto de entrada de execución do programa principal.

Nas primeiras dúas unidades do módulo imos esquecermos do concepto de clase e métodos, centrais na programación en Java, para centrarnos nos tipos de datos primitivos e estruturas de control.

O nome do ficheiro .Java que contén o código fonte do noso programa coincidirá co nome da clase que estamos describindo nestas liñas.

A ter en conta!

Debemos ter en conta que Java distingue entre maiúsculas e minúsculas. Polo tanto se lle déramos á clase principal o nome FirstProgram, o ficheiro .java terá como identificador FirstProgram.Java, que é totalmente diferente de firstprogram.Java.

Ademais, para Java os elementos FirstProgram e firstprogram serían considerados dúas clases diferentes dentro do código fonte.

Imos analizar cada un dos elementos que aparecen no código Java anterior:

- ✓ **public static void main (String[] args):** É o método que representa ó programa principal, pode incluír as instrucións que consideremos oportunas para que a execución da aplicación. A partir de aí pódese utilizar o resto das clases creadas, e



debemos ter en conta que todos os programas Java teñen un método principal.

- ✓ **Comentarios:** Os comentarios adoitan incluírse no código fonte para facer aclaracións, anotacións ou calquera outra indicación que o programador considere oportuno. Estes comentarios pódense introducir de dúas formas:
 - **Con dobre barra ("//"):** Neste caso so aplica á liña na que se atopa, e indica ao compilador Java de bytecode (javac) que non teña en conta esa liña. Un exemplo no anterior código son as liñas: 7, 8 e 9.
 - **Con /* */:** Deste xeito podemos comentar N liñas, as que se atopen entre as chaves de apertura (/*) para o comentario, e o comentario de peche ou fin de comentario (*).
- ✓ **Bloques de código:** son **conxuntos de instrucións que se marcan mediante chaves de apertura e peche { }**. O código marcado deste xeito considérase interno ao bloque. Serve para agrupar de xeito lóxico un conxunto de instrucións baixo unha estrutura concreta.
- ✓ **Punto e coma:** no código hai un exemplo de uso, na liña 18, que se encarga de imprimir por consola a mensaxe "Ola mundo!", e vemos que remata en punto e coma (;). Hai que subliñar que cada liña de código que represente unha instrución Java, debe rematar con punto e coma, pois se non, teremos erros sintácticos e o código non compilará.

6.1 A API de Java

Xunto co kit de desenvolvemento que descargamos e instalamos con anterioridade, inclúense gratuitamente todas as bibliotecas Java API (Application Programming Interface), isto é o que se coñece como Java Class Libraries. Este conxunto de bibliotecas proporciona ao programador paquetes de clases útiles para realizar varias tarefas dentro dun programa. Está organizado en paquetes lóxicos, onde cada paquete contén un conxunto de clases relacionadas semanticamente.

Unha biblioteca de clases é un conxunto de clases de programación orientada a obxectos. Esas clases conteñen métodos que son útiles para os programadores. No caso de Java, cando descargamos o JDK, obtemos a biblioteca de clases API. O uso das clases e métodos das API de Java reduce o tempo de desenvolvemento do programa.

Ademais, existen varias bibliotecas de clases desenvolvidas por terceiros que conteñen



compoñentes de software reutilizables e están dispoñibles na Web a través de múltiples proxectos.

6.2 Compilación e execución por terminal

Unha vez completada a configuración do contorno Java, e temos o código fonte do noso programa escrito nun ficheiro con extensión .Java, a compilación da aplicación realízase mediante o programa Javac incluído no software de desenvolvemento Java.

Para realizar a compilación desde a liña de comandos, escribiremos:

```
javac programa.java
```

Onde Javac é o compilador de Java e programa.java é o noso código fonte.

Se o compilador detecta erros, amosaraos no terminal. Como programadores, teremos que corrixilos para xerar o código obxecto, e volver a compilar a bytecode até que non solte ningún erro e todo vaia ben. Normalmente o compilador ofrécenos información sobre os erros detectados para facilitar a súa corrección.

O resultado da compilación será un ficheiro co mesmo nome que programa.java, pero coa extensión .class, programa.class. Este xa é o ficheiro co código en forma de bytecode. É dicir, co código pre-compilado. Se no código fonte do noso programa aparece máis dunha clase, veremos como ao compilalo se xerarán tantos ficheiros con extensión .class como clases teñamos. Ademais, se estas clases tivesen un método principal, podemos executar estes ficheiros por separado para ver como funcionan estas clases.

Para que o programa se poida executar, sempre que se inclúa o método principal "main" nel, para o que podemos utilizar o intérprete do JDK, desde a liña de comandos, escribindo:

```
java programa
```

Onde java é o intérprete de bytecode, e programa é o ficheiro co código pre-compilado en bytecode que xeramos antes e que ten a extensión .class (programa.class).



7 DATOS E OPERADORES EN JAVA

7.1 Variables

As variables son elementos da linguaxe que permiten que os nosos programas almacenen valores para o seu uso posterior. Para crear unha nova variable (e poder utilizala máis tarde) debemos declarala, e faise co formato ``tipo_de_variable nome_variable``, como nos seguintes exemplos:

```
float tipoDesconto;  
int totalStock;
```

No anterior exemplo, **float** ou **int** definen o tipo da variable e **tipoDesconto** e **totalStock** son os seus identificadores ou nomes.

En función do tipo indicado para a variable, a JVM reservará un número de espazo na memoria para esa variable. Ese nome permanecerá "asociado" a dita localización de memoria.

tipoDesconto



Posición en memoria

O tipo tamén determina a "natureza" dos datos que poden ser almacenados nunha variable (en realidade, nas posicións de memoria "asociadas" á variable) e como se codifican.

Java permite declarar máis dunha variable na mesma instrución, sempre que sexan do mesmo tipo:

```
int numero1, numero2, total;
```

Os nomes das variables deben seguir as regras xerais para calquera identificador en Java:

- Comezan cunha letra, subliñado (_) ou dólar (\$).
- Os seguintes caracteres poden ser letras, díxitos, guión baixo (_) ou dólar (\$)
- As maiúsculas e minúsculas son sensibles
- Non hai unha lonxitude máxima para o identificador

Os **identificadores non poden ser ningunha das palabras reservadas**, que se mostran a continuación:



abstract	default	goto	package	synchronized
assert	do	if	private	this
boolean	double	implements	protected	throw
break	else	import	public	throws
byte	enum	instanceof	return	transient
case	extends	int	short	true
catch	false	interface	static	try
char	final	long	strictfp	void
class	finally	native	super	volatile
const	float	new	switch	while
continue	for	null		

Por convención, a notación **CamelCase** úsase para os identificadores:

- ✓ **UpperCamelCase**: para nomes de clases (que se verán en seguintes unidades).
- ✓ **lowerCamelCase**: para métodos, atributos, variables locais, etc.

Dicimos que unha variable está inicializada se foi declarada e contén algún valor, por exemplo:

```
int numero1 = 23;
```

Unha vez inicializada a variable, podemos usala en posteriores sentenzas do noso programa. A través dela, accederemos ao contido das posicións de memoria que identifica, para ler ou modificar o seu contido.

Para acceder ao contido dunha variable (ler), só temos que facelo usando o seu identificador en calquera expresión ou enunciado, por exemplo:

```
System.out.println("A suma de " + numero1 + " e " + numero2 + " é " +  
(numero1 + numero2));
```

Usando o operador de asignación = (non confundir co dee comparación == que veremos máis adiante), podemos inicializar cun valor ou modificar o valor de calquera variable do seguinte xeito:

```
variable = expresión;
```

- 1) Avalíase a expresión á dereita do símbolo de igual (=).
- 2) O valor resultante gárdase na variable indicada á esquerda.

Java permite inicializar unha variable no momento da súa declaración asignar o valor dun



literal (valor estático) ou dunha expresión (dinámico). Por exemplo:

```
double base = 2.0, altura = 1.2; // estática
double areaR = base * altura; // dinámica
```

7.1.1 Tipos de erro no uso de variables

A lectura de variables non declaradas ou non inicializadas, ou a asignación de un tipo de dato incompatible, xerará erros de compilación:

- **Uso de variables non declaradas:** ao compilar, xerará un erro do símbolo non atopado:

```
int num1 = 5, suma;
suma = num1 + num2;
```

Isto devolverá o seguinte erro:

```
... error: cannot find symbol
      suma = num1 + num2;
                  ^
```

- **Uso de variables non inicializada:** declarada pero sen un valor asignado, cando isto sucede, ao compilar xerará un erro de variable non inicializado:

```
int num1 = 5, num2, suma;
suma = num1 + num2;
```

Isto devolverá o seguinte erro:

```
... error: variable num2 might not have been initialized
      suma = num1 + num2;
                  ^
```

- **Asignación de tipos incompatibles:** No seguinte exemplo, tentamos asignarlle un valor booleano (true/false) a unha variable enteira, que produce un erro de falta de coincidencia de tipos

```
int num1 = true;
```

Isto devolverá o seguinte erro:

```
Exception in ...: Unresolved compilation problem:
      Type mismatch: cannot convert from boolean to int
```



Aínda que o veremos en unidades posteriores, é conveniente saber que as variables teñen un alcance que determina a súa visibilidade e tempo de vida.

7.2 Tipos de datos

Ao desenvolver os nosos propios programas, estes terán que traballar con numerosos datos de distinta natureza: números (números, reais,...), textos, caracteres individuais, lóxicos si/non, etc. Para facilitar o tratamento destes datos, a linguaxe Java, tal como acontece noutros idiomas de alto nivel, ofrece unha serie de tipos de datos para categorizalos e definen como se almacenan internamente eses datos.

Aínda que internamente todos os datos están almacenados en binario, os esquemas de codificación utilizados para a súa representación binaria difiren dependendo da súa propia natureza (ou tipo)

Java, como acontece con outras linguaxes como C++, C# ou Haskell, é de **tipado estático**. Existen outros, como Python ou PHP, escríbense de forma dinámica. As linguaxes deste tipo, normalmente compiladas, obrígnanos a establecer o tipo dunha variable (declaración) antes de usala.

Pola contra, en linguaxes de tipo dinámico, é no proceso de asignación do valor a unha variable cando se define o tipo de datos da mesma.

Ademais, Java é de **tipado forte**, isto implica que a unha variable dun tipo de dado, só se poden asignar valores dese mesmo tipo. O que si permite facer Java, en certos casos, é realizar conversións (implícitas ou explícitas) entre distintos tipos de datos.

Para garantir isto, tanto o compilador Java (no tempo de compilación), como a JVM (en tempo de execución), comprobarán que a manipulación dos datos a través de variables, non viola as especificacións da linguaxe. En función do tipo de datos, en Java podemos facer dúas categorías: primitivos e referenciados.

7.2.1 Tipos primitivos

Representan un único dato simple. Establecen a codificación binaria e intervalo de valores específicos. Son a base de almacenamento en Java e distinguimos os seguintes:

- **byte**: para enteiros (8 bits).
- **short**: para enteiros curtos (16 bits).
- **int**: para números enteiros (32 bits).
- **long**: para enteiros grandes (64 bits).



- **float**: para números reais (precisión única).
- **double**: para números reais (dobre precisión).
- **char**: para caracteres (UNICODE, 16 bits).
- **booleano**: para valores lóxicos (true/false).

7.2.1.1 Números enteiros

Para o almacenamento de números enteiros, Java define os tipos:

TIPO	BITS	RANGO
byte	8	-128 a 127
short	16	-32.768 a 32.767
int	32	-2.147.483.648 a 2.147.483.647
long	64	-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807

Java non admite os enteiros sen asinar (só positivos) como o fan outras lilinguaxes (C, C++, ...). Polo xeral o tipo máis usado para almacenar números enteiros é int, aínda que dependerá de cada uso particular e do rango necesario para a variable concreta.

Co obxectivo de facilitar a portabilidade, os tamaños de almacenamento son fixos e independente da arquitectura (isto aplícase a todos os tipos de datos primitivos).

7.2.1.2 Números en punto flotante

A representación en coma flotante é unha forma de notación científica empregada en ordenadores coa que se poden representar números reais extremadamente grandes e pequenos, dun xeito eficiente e compacto. O estándar actual para a representación binaria de números de coma flotante é o [IEEE 754](#).

Actualmente Java admite dous tipos de números de coma flotante, float e double, que representan números de precisión simple e dobre respectivamente.

TIPO	BITS	RANGO
float	8	Apróx. $\pm 3.40282347E+38F$ (6-7 díxitos decimais significativos)
double	16	Apróx. $\pm 1.79769313486231570E+308$ (15 díxitos decimais significativos)

7.2.1.2.1 A precisión en números en punto flotante

Temos que ter en conta que, tanto o tipo **float** como o **double**, teñen un número limitado



de bits para representar a mantisa e o expoñente dos nosos números reais. Por iso na meirande parte dos casos, serán aproximacións a ditos números. Por exemplo, se temos a seguinte sentenza:

```
System.out.println(0.3 + 0.6);
```

O resultado de execución da instrución, de xeito sorprendente, será:

```
0.8999999999999999
```

en lugar do resultado esperado:

```
0.9
```

Isto é porque non hai representación exacta de dito valor en [formato IEEE 754](#) (necesitaríamos un número infinito de bits). Do mesmo xeito, se executamos:

```
double n1=0.3, n2=0.6, suma = n1 + n2;  
System.out.println("A suma de n1 e n2 dará 0.9? " + (suma == 0.9));
```

O resultado sería:

```
A suma de n1 e n2 dará 0.9? falso
```

Isto pode dar lugar a certas condutas que non o son desexado, especialmente en aplicacións financeiras, onde traballamos cun número fixo de cifras decimais.

Para solucionar este problema, nas vindeiras unidades, veremos que Java ofrece a clase [BigDecimal](#) que permite definir o número de díxitos de precisión para os decimais, así como o tipo de redondeo cando se traballa con reais.

7.2.1.3 Desbordamentos

Vimos como as variables, segundo o seu tipo, teñen un espazo máximo reservado na memoria e, polo tanto, un rango de valores posible. Mais, que pasa se nos saímos dese rango. Supoñamos que tentamos asignar a a variable de tipo int un valor fóra do seu intervalo [-2.147.483.648 a 2.147.483.647]:

7.2.1.3.1 Desbordamento en compilación

No caso de que se realice unha asignación fora de rango no propio código mediante un valor literal, será o compilador o que marque o erro cando compilemos ese código. Por exemplo:



```
int n = 2147483650;
```

Neste caso o compilador informaranos desta situación e abortará a compilación:

```
...: error: integer number too large: 2147483650
      int n = 2147483650;
              ^
1 error
```

7.2.1.3.2 Desbordamento en execución

Se o desbordamento ocorre en tempo de execución que, por exemplo, pode suceder se a unha variable de tipo byte, cun rango [-128, 127] lle asignamos un valor superior:

```
byte b = 127; // Asignamos o valor máis alto do intervalo
b += 1; // Aumentamos o seu valor en 1
System.out.println(b);
```

Aquí, ao contrario do que poderíamos esperar, non se produce un erro de execución e obtemos como resultado o número -128.

Para evitar estes erros de desbordamento, que poderían causar unha corrupción das áreas de memoria adxacentes, pois **os rangos de valores primitivos dos tipos de datos en Java son circulares**. É dicir, cando superamos o valor máximo coñecido como **overflow**, entón continua polo extremo inferior do rango, e viceversa, que se coñece como unha situación de **underflow**. Por este motivo é moi importante telo presente ao operar con valores numéricos

7.2.1.4 Caracteres

Java proporciona o tipo char para o almacenamento de caracteres, que se representan mediante codificación **UNICODE**, de xeito que se pode usar calquera carácter das diferentes linguaxes humanas de comunicación.

A diferenza doutras linguaxes, como C ou C++, nos que 8 bits son suficientes para almacenar as representacións ASCII dos caracteres, a utilización de **UNICODE** implicou un tamaño superior, neste caso o tipo char precisa de 16 bits, que permite almacenar valores numéricos enteiros positivos no intervalo de 0 a 65535, no que cada número representa un carácter concreto. É dicir, o tipo char é un tipo numérico, polo que poden ser realizar operacións aritméticas sobre ela (suma, resta,...).

Os literais, ou valores que se asignan, irán entre comiñas simples. Por exemplo: 'a', '4', '\



n', etc. A continuación temos un exemplo:

```
public class Exemplo {  
    public static void main(String[] args) {  
        char ch = 'X'; // Declaramos 'ch' e asignamos o carácter 'X'  
        System.out.println(" - 0 carácter contén " + ch);  
  
        ch += 1; // Aumentamos o seu valor actual en 1  
        System.out.println(" - 0 carácter agora contén " + ch);  
  
        //As variables de carácter pódense tratar como números enteiros  
        ch = 90; // Asignamos un valor numérico  
        System.out.println(" - 0 carácter agora contén " + ch);  
  
        /**  
         * Saída:  
         * - 0 carácter contén X  
         * - 0 carácter agora contén Y  
         * - 0 carácter agora contén Z  
         */  
    }  
}
```

7.2.1.5 Valores lóxicos

O tipo booleano representa valores lóxicos true/false. En Java, as palabras **true** e **false** son palabras reservadas utilizadas para establecer os dous únicos valores posibles dunha variable booleana. Os usos típicos do tipo booleano son:

- O resultado de calquera operación que utilice operadores relacionais (>, <, >=, ==, ...) serán de tipo booleano.
- Control de instrucións condicionais, que veremos en detalle na seguinte unidade do módulo (**if**).
- Creación de variables de dominio binario (**flags**): true/false, on/off, si/non, etc. Por exemplo: pedidoEnviado, usuarioValidado, etc.

7.2.2 Tipos por referencia

Cando o tipo da nosa variable é unha clase, matriz ou interface, é dicir, un tipo que non sexa primitivo, entón a variable correspondente almacenará un “endereço memoria” (referencia) a un obxecto ou conxunto de valores. Con este tipo de datos traballaremos máis polo miúdo nas vindeiras unidades.



7.3 Literais

En xeral, o termo literal refírese a valores fixos representados para que poidan ser lidos por nós. Por exemplo, o número 100, ou o texto "aquí temos un exemplo dun literal". Polo tanto usamos literais para facer asignacións a variables e constantes (variables cuxo valor non se pode modificar).

7.3.1 Literais de tipos primitivos

Podemos crear literais de calquera dos tipos primitivos. A forma de representalos dependerá do seu tipo:

- **Caracteres:** entre comiñas simples. Por exemplo: 'a', '3', '%'.
- **Números enteiros:** en forma numérica (con ou sen signo). Por exemplo: 9, -3.
- **Números en coma flotante:** co punto decimal (non coma). Por exemplo: 3.5, -128.2. Neste caso tamén se pode usar a notación científica: 2.3e4, 1.25e-2.

A ter en conta!

Desde a versión JDK7, podemos inserir un ou máis guións baixos (_) entre os díxitos dun número enteiro ou de coma flotante, e que non ten en conta. Isto pode facelo máis doado ler números de varios díxitos como referencias de pedidos, etc.

Podemos ver un exemplo do que indico:

```
long decimal = 24_09_12_012;  
System.out.println(" - Decimal vale " + decimal);
```

O resultado sería:

```
- Decimal vale 240912012
```

Por defecto, os literais enteiros son de tipo int e pódense asignar directamente a variables de tipo char, byte ou short sempre que o valor non exceda o rango da variable. Debemos ter en conta tamén que Un literal enteiro sempre se pode asignar a unha variable de tipo long.

Podemos crear un literal longo engadindo unha l (ou L) ao número, por exemplo:

```
long numero1 = -125l;  
long numero2 = 24L;
```

Por defecto, os literais de coma flotante son do tipo double, así que se desexamos que



sexa de tipo float podemos facelo de xeito explícito engadindo unha f (ou F) ao número:

```
float numero1 = 125.56f;  
float numero2 = 24.23F;
```

Ás veces é máis recomendable expresar un valor numérico en base 8 ou 16 en lugar da base 10. Por exemplo: enderezos IPv6, posicións de memoria, compoñentes RGB de cores, etc. Java permítenos expresar literais de tipo enteiro usando notación octal ou hexadecimal en lugar de decimal. Podemos facelo do seguinte xeito:

- **Octal:** prefixando un 0 ao valor octal: 011 (9 en decimal).
- **Hexadecimal:** prefixando 0x (ou 0X) ao seu valor hexadecimal.
- **Binario:** é posible expresar un literal enteiro mediante a notación binario. Para iso, precederemos o número binario con 0b (ou 0B).

A continuación podemos ver un exemplo:

```
int numero1 = 0xFF;  
int numero2 = 011;  
int numero3 = 0b1100;  
  
System.out.println(" - Numero 1: " + numero1);  
System.out.println(" - Numero 2: " + numero2);  
System.out.println(" - Numero 3: " + numero3);
```

Isto mostraría por pantalla o seguinte:

```
- Numero 1: 255  
- Numero 2: 9  
- Numero 3: 12
```

7.3.2 Literais de tipo String

Java admite outro tipo de literal que é a cadea, trátase dunha secuencia de caracteres entre comiñas dobres, por exemplo: "esta é unha proba"

Ademais dos caracteres alfanuméricos, a cadea pode incluír caracteres especiais tamén coñecidos como caracteres de escape, que van precedidos da barra invertida (\) e teñen un significado particular. Por exemplo:

SECUENCIA DE ESCAPE	SIGNIFICADO
\n	Nova liña
\r	Retorno de liña



\f	Avance de liña
\t	Tabulación
\b	Retroceso
\"	Comiñas dobre
\'	Comiñas simple
\\	Barra invertida

As secuencias de escape, como caracteres que son, poden asignarse individualmente a variables de tipo char. A continuación vemos un exemplo de uso:

```
/**
 * Exemplos de uso de secuencias de escape
 */
public class EscapeDemo {

    public static void main(String[] args) {
        char c = '\t'; // secuencia de tabulacions
        System.out.println("\"Isto vai\nen varias liñas entre aspas\"");
        System.out.println("As\tseguintes\tpalabras\ttestán\ttabuladas");
        System.out.println("E" + c + "estes" + c + "tamén");
    }
}
```

7.4 Constantes

As constantes son un tipo especial de variables que, unha vez lles asignamos o seu primeiro valor, xa non se pode modificar e permanecerá inmutable. A declaración de constantes é semellante á de calquera variable, coa diferenza que simplemente será preciso engadir o modificador **final** ao comezo do mesmo. Por exemplo:

```
final float TIPO_IVA = 0.21f;
final double PI = 3.141592;
```

O modificador final fai imposible que as variables sexa modificado despois da inicialización, pois produciría un erro de compilación. En posteriores unidades veremos tamén que ese modificador pode ser aplicado a clases e métodos.

Por convención, os nomes das constantes van en maiúscula e en nomes compostos, empregarase o guión baixo (_) para separar as palabras. Este tipo de técnicas axudan a que o código sexa máis lexible e mantible.



7.5 Operadores

Os operadores son símbolos que usamos para indicar a realización dalgún tipo de manipulación aritmética ou lóxica sobre os datos. En Java temos as seguintes clases de operadores:

- **Aritméticos:** +, -, *, etc.
- **Relacionais:** >, >=, <, etc.
- **De bit:** >>, <<, etc.
- **Lóxicos:** &, &&, ||, etc.
- **De asignación:** =, +=, -=, etc.
- **Operador ternario** (?:) e **tipo de instancia** (instanceOf) que se verán ambos na unidade 3.

7.5.1 Operadores aritméticos

Os operadores aritméticos utilízanse en expresións matemáticas do mesmo xeito que se usan en álgebra

OPERADOR	SIGNIFICADO
+/-	Signo (positivo/negativo)
+	Suma
-	Resta
*	Multiplicación
/	División
%	Módulo (devolve o resto da división enteira indicada)
++	Incremento (+1)
--	Diminución (-1)

O operador - permítenos modificar o signo dunha variable:

```
int num = 3, num2;  
num2 = -num1; // num2 é igual a -3
```

O operador módulo % devolve o resto dunha división enteira:

```
int num1 = 3, num2 = 8, resto;  
resto = num2 % num1; // resto é igual a 2
```



Os operadores ++ (incremento) e -- (diminución) permítennos aumentar ou diminuír o valor actual dunha variable nunha unidade, algo moi común, por exemplo, no control dun bucle, estrutura que veremos na seguinte unidade:

```
int num1 = 1; // num1 é igual a 1
num1++; // num1 é igual a 2 equivalente a: num1 = num1 + 1
```

A instrución n++ anterior sería equivalente a realizar a tarefa:

```
num1 = num1 + 1;
```

Pódense usar ambos operadores de incremento (++) e diminución (--) en forma de prefixo (++n), cou omo sufixo (n++), aínda que con resultados diferentes.

- **Como prefixo:** o valor da variable actualizarase primeiro e entón o valor actualizado empregarase na expresión:

```
int num1 = 10, num2;
num2 = ++num1; // num1 é 11 e num2 é 11
num2 = ++num1 * 2; // num1 é 12 e num2 é 24
```

- **Como sufixo:** a expresión avaliarase primeiro co valor actual da variable e, posteriormente actualizarase o valor da mesmo:

```
int num1 = 10, num2;
num2 = num1++; // num1 é 11 e num2 é 10
num2 = num1++ * 2; // num1 é 12 e num2 é 22
```

7.5.2 Operadores relacionais

Os operadores relacionais permítennos construír expresións para comparar valores.

OPERADOR	SIGNIFICADO
==	Igual que
!=	Distinto que
>	Maior que
>=	Maior ou igual que
<	Menor que
<=	Menor ou igual que

O resultado da expresión será sempre un valor booleano (true/false)



7.5.3 Operadores lóxicos

Os operadores lóxicos permítennos crear expresións lóxicas complexas, normalmente formado por expresións que inclúen operadores relacionais.

OPERADOR	SIGNIFICADO
&	AND (E lóxico)
	OR (Ou lóxico)
^	XOR (Ou exclusivo)
	OR (Short-circuit). Se cumpre o primeiro xa non comproba o segundo parámetro
&&	AND (Short-circuit). Se non cumpre o primeiro xa non comproba o segundo parámetro
!	NOT (negación)

Tanto os operandos, como o resultado da expresión lóxica, serán valores booleanos (true/false). A continuación mostramos as táboas de verdade dos diferentes tipos de operadores lóxicos:

& (AND)			(OR)			^(XOR)		
Operandos		Resultado	Operandos		Resultado	Operandos		Resultado
True	True	True	True	True	True	True	True	False
True	False	False	True	False	True	True	False	True
False	True	False	False	True	True	False	True	True
False	False	False	False	False	False	False	False	False

! (NOT)	
Operando	Resultado
True	False
False	True

Java ofrece versións de **short-circuit** dos operadores lóxicos AND (&) e OR (|), que se poden usar para xerar código máis eficiente. Mentres cos operadores estándar AND (&) e OR (|) sempre se avalía cada un dos operandos dunha expresión lóxica, as súas versións short-circuit, avaliarán o segundo operando só no caso de ser necesario:

- **Operador AND short-circuit é &&:** Se o primeiro operando é falso, o segundo xa non se avalía, xa que o resultado da expresión sempre será false.



- **Operador OR de short-circuit é ||:** Se o primeiro operando é verdadeiro, o segundo xa non se avalía, xa que o resultado da expresión sempre será true.

7.5.4 Operadores bitwise

Os operadores bitwise pódense aplicar a calquera valor enteiro numérico: byte, char, short, int, long. Permite realizar operacións lóxicas a nivel de bit cos valores que representan eses números en binario.

OPERADOR	SIGNIFICADO
&	AND (E binario)
	OR (Ou binario)
^	XOR (Ou exclusivo binario)
<<	LS (Desprazamento esquerdo)
>>	ARS (Desprazamento dereito aritmético, preserva signo)
>>>	LRS (Desprazamento dereito lóxico, recheo con 0's)
~	COMP (complementario)

No [seguinte link](#) temos exemplos detallados do uso destes operadores sobre valores numéricos.

7.5.5 Operadores de asignación compostos

O operador de asignación de Java (=) ten un comportamento intuitivo que é o máis habitual e o que vimos até o de agora, mas tamén conta con outras características:

- **Cadeas de asignación:** podemos realizar unha asignación múltiple a un grupo de variables:

```
int a, b, c;  
a = b = c = 33; // a, b e c valen 33
```

- **Operadores de asignación compostos:** Usando un único operador, podemos combinar unha operación aritmética e unha asignación, mellorando así a lexibilidade do código.

```
int a = 33; // a vale 10  
a += 7; // a é igual a 40 (é equivalente: a = a + 7; )  
a /= 4; // a é igual a 10 (é equivalente: a = a / 4; )
```



A continuación detallamos o conxunto de operadores de asignación compostos que podemos utilizar en Java.

OPERADOR	SIGNIFICADO
<code>+=</code>	Suma
<code>-=</code>	Resta
<code>*=</code>	Multipliación
<code>/=</code>	División
<code>%=</code>	Módulo
<code><<=</code>	Left-shift
<code>>>=</code>	Right-shift
<code>>>>=</code>	Logical-right-shift
<code>&=</code>	Bitwise AND
<code> =</code>	Bitwise OR
<code>^=</code>	Bitwise XOR

7.5.6 Precedencia de operadores

A asociatividade e precedencia dos operadores establecen o xeito no que se avalían as expresións, especialmente cando están implicados varios operandos e operadores.

Se temos:

```
8 + 3 - 5 = 6
```

A suma e a resta teñen a mesma precedencia e están asociadas á esquerda, polo que a orde na que se resolvería sería: primeiro $8+3$ e, do resultado, resta 5.

No seguinte exemplo:

```
7 + 3 * 4 = 19
```

O produto ten maior precedencia que a suma e asóciase pola esquerda, así que a orde na que se resolvería sería: primeiro $3*4$ e, ao resultado, súmalle 7.

Agora, se facemos:

```
(5 + 3) * 4 = 32  
(8 + 2) * (7 - (3 + 2)) = 20
```



O uso de parénteses permítenos modificar a orde na que se avalía a expresión, desde os parénteses máis internos aos máis externos.

A continuación mostramos unha táboa resume coa orde de precedencia, de menor a maior:

CATEGORÍA	OPERADOR	ASOCIATIVIDADE
Postfix	Expr [++ --]	Esquerda → Dereita
Unario	[++ -- + - ~ !] Expr	Dereita → Esquerda
Multiplicativo	* / %	Esquerda → Dereita
Aditivo	+ -	Esquerda → Dereita
Relacional	< > <= >= instanceof	Esquerda → Dereita
Igualdade	== !=	Esquerda → Dereita
Bitwise AND	&	Esquerda → Dereita
Bitwise XOR	^	Esquerda → Dereita
Bitwise OR		Esquerda → Dereita
AND	&&	Esquerda → Dereita
OR		Esquerda → Dereita
Condiciona ternario	? :	Dereita → Esquerda
Asignación	= += -= *= /= %= ^= = <<= >>= >>>=	Dereita → Esquerda

As parénteses úsanse para agrupar termos en expresións en Java, do mesmo xeito que nas expresións alxébricas. Por exemplo, para multiplicar a por b + c, escribimos:

```
a*(b+c)
```

De non facelo deste xeito, debido á maior precedencia da multiplicación, En canto á suma, primeiro faríase a * b e despois sumaría c ao resultado.

Se unha expresión contén parénteses aninadas, como:

```
a / ((b + c) * d)
```

A expresión avalíase primeiro no conxunto máis interno:

```
a
```



```
(b + c) × d
```

7.6 Conversións de tipos

En programación, é común asignarlle o valor dunha variable dun tipo a outra variable de diferente tipo. Por exemplo:

```
int num1 = 33;  
float flot1;  
flot1 = num1 // asignación dun int a un flotante flot1 é 33,0
```

Cando usamos tipos compatibles, o valor da dereita pasa a ser automaticamente o tipo da esquerda. No exemplo anterior, o valor de num1 convértese en flotante e despois asígnase a flot1. De todos os xeitos, debido á natureza fortemente tipificada de Java, estas conversións implícitas non sempre son posible. Por exemplo, os tipos boolean e int non son compatibles.

Permitirase unha conversión automática de tipos se:

- Os tipos implicados son compatibles
- O tipo de destino é maior que o tipo de orixe

7.6.1 Conversión de ampliación (widening-conversion)

Nas operacións aritméticas nas que interveñen datos de diferentes tipos, os tipos ascenden ao operando de tipo máis grande:

- byte --> short, int, long, float, double
- short --> int, long, float, double
- char --> int, long, float, double
- int --> long, float, double
- long --> float, double
- float --> double

A continuación vemos unha serie de exemplos de conversións:

```
int num = 5;  
float dec;  
dec = 22,0/num; // dec vale 4,4
```

No exemplo anterior, o divisor (int 5) ascende a flotante (5.0), que é o tipo do dividendo



(22,0), antes de realizar a operación que dá como resultado un número en coma flotante, neste caso 4,4.

7.6.2 Conversión de tipos incompatibles (casting)

Aínda que as conversións automáticas son útiles, haberá situacións nas que non se cumpran as condicións de ampliación. Por exemplo:

```
int ent;  
float flot = 33,47F;  
ent = flot; // erro de compilación
```

Isto xerará un erro de compilación ao tentar asignar o valor no punto float é variable de tipo enteira:

```
...: error: incompatible types: possible lossy conversion from float to int  
    ent = flot;  
        ^
```

Neses casos, teremos que "forzar" ao compilador para que realice a conversión, e iso farase invocando unha operación de **cast** coa seguinte sintaxe:

```
(tipo_requirido) expr
```

Onde o tipo requirido, entre parénteses (), será o tipo de datos ao que queremos converter o resultado da avaliación da expresión. Por exemplo:

```
int ent;  
float flot = 33,47F;  
ent = (int) flot; // cast a int de flot entón ent será 33
```

Temos que ter en conta que estas conversións reducindo, poden provocar a perda de información. Por exemplo, a conversión dun valor de coma flotante a un tipo enteiro, sempre asumirá o truncado da parte decimal (no exemplo perderíamos o 0,47). Pódese facer cast sobre calquera expresión que devolva un valor, por exemplo:

```
(int)(x/y)
```

No exemplo anterior debemos fixarnos nos parénteses que rodean a expresión x/y, pois de non usalos o cast só se aplicaría á variable x.



No seguinte código temos un exemplo con varias conversións deste tipo:

```
/**
 * Exemplos de conversións
 */
public class CastDemo {

    public static void main(String[] args) {

        byte byte1;
        char ch1;
        int int1;
        double dec1, dec2;

        dec1 = 30.0;
        dec2 = 5.0;
        int1 = (int) (dec1 / dec2); // Truncado do resultado
        System.out.println("A división x/y é " + int1);

        int1 = 89;
        byte1 = (byte) int1; // Non hai perdas (89<127)
        System.out.println("O valor de b é " + byte1);

        int1 = 323;
        byte1 = (byte) int1; // Un byte non pode almacenar 323!!!
        System.out.println("O valor de b é " + byte1);

        int1 = 107; // ASCII de 'k'
        ch1 = (char)int1; // Non hai perdas (107<65535)
        System.out.println("O valor de ch é " + ch1);    }

}
```

7.6.3 Normas de reparto e promoción de tipos

Cada vez que se avalía unha expresión, aplícanse as regras de promoción de tipos de Java:

- 1) Os valores de char, bytes e short promóvense a int.
- 2) Se algún dos operandos é long, a expresión promóvese a long.
- 3) Se algún dos operandos é float, a expresión promóvese a float.
- 4) Se algún dos operandos é double, promóvese a double.



En base ao indicado anteriormente, existirán situacións nas que nos vexamos obrigados a realizar un cast aínda que os operandos e o destino sexan do mesmo tipo. Por exemplo:

```
int ent1;  
byte byte1 = 10;  
ent1 = byte1 * byte1; // non é necesario lanzar  
ent1 = (byte) (byte1 * byte1); // facemos cast (o resultado sáese do rango)
```

8 API DE JAVA

Calquera linguaxe moderna de alto nivel pon á disposición do programador unha extensa biblioteca de utilidades e ferramentas para facilitar o seu traballo. Unha parte esencial destas ferramentas son as API (Application Programming Interface). Trátase dun conxunto de subrutinas, funcións e procedementos (clases e métodos na programación orientada obxectos) que se pon a disposición dos programadores para a realización de tarefas habituais. Por exemplo:

- Entrada/saída de datos por consola, ficheiros, rede, etc.
- Cálculos matemáticos complexos.
- Tratamento de erros en tempo de execución.
- Xestión de estruturas de datos avanzadas: listas, colas, árbores, mapas etc.
- Deseño de GUI (Interface gráfica de usuario).

O número de clases que compoñen a biblioteca Java é enorme, é por iso que están organizados en diferentes paquetes en función da súa aplicación. A documentación oficial da [API de Java está dispoñible tanto en liña](#) como para [descargar](#), e **debe ser o recurso fundamental dun programador Java**.

Facer uso destas clases (e dos seus métodos) en Java é sinxelo, e será algo que veremos en profundidade nas vindeiras unidades e que se realizará na seguinte orde:

- 1) **Importar a clase:** excepto as do paquete java.lang, que se importan automaticamente. Por exemplo: java.lang.Math, java.lang.System,...
- 2) **Invocar ao método da clase:** se é estático (por exemplo, Math.sqrt), ou senón crear un novo obxecto desa clase para chamar ao método (por exemplo, nextInt de Scanner).



8.1 Saída por consola

Unha das operacións máis básicas que proporciona a API é a que nos permite mostrar mensaxes de texto na consola do sistema. Na unidade 3 veremos en profundidade que é unha clase, como se define e como se poden utilizar as súas propiedades, aínda que para poder comezar a programar, debemos poder mostrar ou inserir información por consola para poder interactuar coas nosas aplicacións dun xeito doado.

Para poder realizar operación de saída pola consola dispoñemos da clase `System` que proporciona acceso a diferentes recursos do sistema. Un dos seus atributos, **`out`**, está "conectado" á **saída estándar** do sistema (polo xeral a pantalla ou a consola), de xeito que nos ofrece unha serie de métodos que nos permiten escribir nela. Os máis utilizados son os seguintes:

- **`System.out.print(mensaxe)`**: mostrará a cadea de texto da mensaxe indicada.
- **`System.out.println(mensaxe)`**: mostrará a cadea de texto da mensaxe indicada e engadirá un salto de liña ao final.
- **`System.out.printf(mensaxe_con_formato, lista_valores)`**: permite realizar unha saída formatada, de xeito que a cadea incluíra unha serie de campos que serán substituídos polos valores da `lista_valores`.

A consola usa un cursor que determina a primeira posición a partir da que se vai imprimir unha nova mensaxe. A medida que se imprimen novos caracteres, este cursor móvese automaticamente ata que estea despois do último carácter impreso.

Vimos anteriormente como utilizar as secuencias de escape, que se poden incluír nos nosos textos, e que alteran a forma no que se imprimen estas mensaxes ao modificar a posición do cursor. Unha destas secuencias de escape, salto de liña ("`\n`") adoita ir ao final das mensaxes, para que así a nova impresión comece na primeira posición da seguinte liña. Por este motivo, a clase Java `System` inclúe tanto `print( )` como `println( )`, pois este último inclúe xa o salto de liña final.

O obxecto `String` das mensaxes pode crearse a partir de literais de tipo cadea caracteres e variables, concatenadas mediante o operador `(+)`. Con independencia do seu tipo, Java obterá unha representación "textual" dos valores das variables para poder imprimilos.



Por exemplo:

```
int idade = 33;  
System.out.println("Eu teño " + idade + " anos");
```

Java interpreta as cadeas "Eu teño" e "anos", entre comiñas, como valores literais e idade (sen comiñas) como variable, de xeito que Java converte o seu valor en texto e concaténalos.

O método `System.out.printf` (f significa formato) permítelle mostrar datos cun formato específico (número de cifras decimais, caracteres de recheo,...). Por exemplo:

```
System.out.printf("Ola %s!\nContinuamos debaixo.", "Mundo");
```

A saída da declaración anterior sería:

```
Ola Mundo!  
Continuamos debaixo.
```

O primeiro argumento do método `printf`, no noso caso "Ola %s!", é unha cadea que pode incluír texto e secuencias de formato (ou campos substitución). Cada unha destas secuencias será substituída no texto impreso por un valor ou comportamento. Por exemplo, %s substitúese pola cadea "Mundo", mentres que %n provoca un salto de liña. As secuencias de formato teñen a seguinte sintaxe:

```
%[flags][ancho][.precisión]carácter_de_conversión;
```

Na seguinte táboa temos un resumo dos diferentes caracteres dispoñibles neste caso:

CARÁCTER	SAÍDA POR CONSOLA
c	Carácter individual
C	Carácter individual (maiúsculas)
b	Booleano
B	Booleano (maiúsculas)
d	Número enteiro
f	Número en punto flotante
e	Número en punto flotante (notación científica)
s	Cadea de caracteres
S	Cadea de caracteres (maiúsculas)



n	Salto de liña
tM	Data e hora (M modificador de formato)

Aquí temos algúns exemplos:

```
boolean bool = true;
char ch = 'x';
String st = "Texto de exemplo"; // Creación dun obxecto String
System.out.printf("Booleano: %b, char: %c e cadea: %s\n", bool, ch, st);
System.out.printf("En maiúsculas: %B, %C e %S", bool, ch, st);
```

A saída diso sería:

```
Booleano: true, char: x e cadea: Texto de exemplo
En maiúsculas: TRUE, X e TEXTO DE EXEMPLO
```

Neste caso tamén se podería indicar o salto de liña co parámetro %n, para o que non faría falta especificar un parámetro na lista. Por exemplo, poderíamos ter algo así:

```
System.out.printf("Boolean: %b, char: %c y string: %s\n", bool, ch, st);
```

Tamén podemos establecer directamente o emparellamento entre o campo de formato e un valor da lista. Para iso, indicaremos no campo de formato a posición ocupada na lista polo valor que nos interesa (a partir da 1). Por exemplo:

```
System.out.printf("%1$b %2$c %1$B %2$C", (5>2), 'x');
```

A saída nesta caso será:

```
true x TRUE X
```

Usaremos a secuencia de formato %d para imprimir valores enteiros (byte, char, short, int, long), mentres para os números de coma flotante usaremos os códigos %f e %e (notación científica). Ademais, podemos especificar a cantos díxitos decimais queremos redondear o valor co flag punto (.) e tamén temos o flag coma (,) se queremos mostrar o separador de miles:

```
System.out.printf("%d %f %n", 123, 6.327); // enteiro e punto flotante
System.out.printf("%1$f %1$.2f %n", 321.856); // sen e con redondeo
System.out.printf("%,.2f %n", 23467.768); // redondeo e separador de miles
System.out.printf("%.2e %n", 123.456); // redondeo e notación científica
```

A saída será:



```
123 6,327000
321,856000 321,86
23.467,77
1,23e+02
```

Por defecto, como separadores de decimais, de miles, ou signos utilizaranse os establecidos pola configuración rexional do sistema. Java ofrece unha variante de `printf` na que podemos pasalo como argumentos a configuración rexional que queremos usar. Isto xa é un pouco máis complexo, pois teremos que usar un obxecto da clase `Locale`, conceptos que veremos en detalle na unidade 3.

A continuación deixamos un exemplo de como se podería modificar esa configuración rexional. Para iso crearemos un obxecto da clase [java.util.Locale](https://docs.oracle.com/javase/8/docs/api/java/util/Locale.html) coa configuración local desexado, ou empregaremos un dos que xa están predefinidos.

```
import java.util.Locale;
...
System.out.printf("df: %,f %n", 34566.984); // config. reg. sistema
(defecto)
System.out.printf(Locale.US, "US: %,f %n", 12983.276); // aplicar a
configuración. reg. EE.UU.

// predefinido en Locale
Locale locFr = Locale.FRANCE; // creamos a configuración. rexional france
System.out.printf(locFr, "FR: %,f %n", 3456.023); // aplicar a config. FR
```

A saída será:

```
df: 34.566,984000
US: 12,983.276000
FR: 3 456,023000
```

Finalmente, tamén temos secuencias de formato para valores de datas e horas, permitíndonos extraer campos individuais.

CARÁCTER	SAÍDA POR CONSOLA
tH	Hora
tM	Minutos
tS	Segundos
tp	am/pm
tz	Zona horaria
tA	Día da semana (texto)



td	Día do mes (numérico de 2 díxitos)
tB	Mes (texto)
tm	Mes (Numérico de 2 díxitos)
tY	Ano (4 díxitos)
ty	Ano (2 díxitos)

Exemplo de uso:

```
import java.time.ZonedDateTime;

/**
 * Exemplos de saídas de formato de data
 */
class PrintDateDemo {
    public static void main(String[] args) {
        // Imos obter datos/hora actual con información da zona
        ZonedDateTime data = ZonedDateTime.now();
        System.out.printf("Hoxe é %1$tA, %1$td de %1$tB de %1$tY %n", data);
        System.out.printf("É %1$tH:%1$tM:%1$tS [%1$tp] %n", data);
        System.out.printf("Na zona horaria %s [%tz] %n", data.getZone(), data);
    }
}
```

A saída sería similar a esta:

```
Hoxe é luns, 23 de setembro de 2024
É 12:21:40 [p.m.]
Na zona horaria Europe/Madrid [+0200]
```

8.2 Entrada por consola

Aínda que temos diferentes alternativas en Java para ler o entrada de datos estándar, unha das formas máis sinxelas é a través do [java.util.Scanner](#), xa que nos permite obter directamente os valores de tipos primitivos sen necesidade de realizar ningunha conversión.

Para poder utilizar calquera dos métodos de escáner, debemos primeiro crear un obxecto dela. Algúns dos seus métodos son:

- **next()**: le a primeira palabra dun String (até que atopa o primeiro espazo).
- **nextBoolean()**: le un valor boolean introducido polo usuario.
- **nextByte()**: le un valor de tipo byte.
- **nextDouble()**: le un valor de tipo double.



- **nextFloat()**: le un valor de tipo float.
- **nextInt()**: le un valor de tipo int.
- **nextLine()**: le toda a entrada como un String.
- **nextLong()**: le un valor de tipo long.
- **nextShort()**: le un valor de tipo short.

A chamada a calquera dos métodos de Scanner é bloqueante, isto quere dicir que o programa detense ata que premamos a tecla [Return], momento no que o control será devolto ao programa e pasaráselle o que introduciu o usuario.

Se ao chamar a algún destes métodos, o programa recibe un dato que non é convertible ao tipo esperado, producirase un erro, e rematará a súa execución. Nas vindeiras unidades veremos como podemos xestionar estes erros de xeito controlado.

Ao introducir valores de coma flotante, Java espera que o fagamos usando a configuración rexional do sistema para separar os decimais (Os literais do programa usan o punto como separador decimal). Podemos modificar este comportamento usando o método `useLocale()` e establecer a configuración rexional para a entrada que nos interese.

Exemplo de uso:

```
/**
 * Exemplos de entrada de datos por consola coa clase Scanner
 */
public class ScannerDemo {
    public static void main(String[] args) {
        // Creamos o obxecto Scanner
        Scanner entrada = new Scanner(System.in);

        // Cambiamos a configuración rexional para usar '.' decimal
        // en lugar de ',', decimal (opcional)
        entrada.useLocale(Locale.US);
        System.out.println("Introduce o teu nome, idade e altura
(metros):");
        String nome = entrada.nextLine();
        int idade = entrada.nextInt();
        double altura = entrada.nextDouble(); // usamos o separador
decimal segundo a configuración rexional
        System.out.println("Nome: " + nome);
        System.out.println("Idade: " + idade);
        System.out.println("Altura (m): " + altura);

        entrada.close();
    }
}
```



```
}
```

Outro dato importante que debemos ter en conta é que despois de ler calquera dato, cando prememos a tecla [Return] a clase Scanner garda esa posición co seu valor correspondente (`\n`), e isto pode ter un comportamento inesperado no noso sistema, por este motivo cando lemos valores numéricos é importante limpar o noso buffer coa instrución `nextLine()` que devolverá ese valor, co que ademais non faremos nada. Temos máis información deste comportamento no [seguinte link](#).